

IOCL GRADE-A 2026

CS/IT WORKBOOK

DAY 3 — BCNF, DESIGN THEORY & ER MODEL

21 AUGUST 2026

Topic: Armstrong's Axioms, Minimal Cover, BCNF, Lossless Decomposition & Dependency Preservation (in depth), 4NF/5NF (awareness level), Advanced ER Model & ER-to-Relational Mapping

Target session length: $\approx$ 2 hours 45 minutes – 3 hours
Exam date: 24 September 2026
Study date: 21 August 2026
Follows on from: Day 2 – FDs & Normalization (1NF–3NF), 20 August 2026
Leads into: Day 4 – Relational Algebra, SQL, Transactions, Indexing

Goal: Comfortable CBT clearance – high-yield Easy/Medium level MCQs, with a smaller number of useful Hard questions. This is *not* GATE-depth mastery.

Day 3 covers more ground than Days 1–2 because BCNF, lossless/dependency-preservation reasoning, and ER design theory naturally need Armstrong's axioms and minimal cover as prerequisites – budget extra time today.

A note on Previous-Year Questions (PYQs) in this workbook:

Label	Meaning
[VERIFIED GATE PYQ –]	A real GATE question, checked against a documented source and independently re-solved here.
[VERIFIED IOCL PYQ –]	A real IOCL previous-year question with reliably verified provenance.
[IOCL-REPORTED SOURCE-UNVERIFIED]	/ Reported as an IOCL question by a coaching site, but its exact IOCL origin could not be independently confirmed.
[PSU/IOCL-PATTERN PRACTICE] / [GATE-STYLE PRACTICE]	An original question written in a PSU/GATE-style pattern – not a real previous-year question.

This is a study diagnostic tool, not an official IOCL cutoff predictor.

A dedicated, honest search for genuine IOCL CS/IT previous-year DBMS questions was carried out for this workbook – see the note at the start of Section 2 for what was and was not found.

Contents

1	Learning Objectives	2
2	Concepts and Mental Models	3
2.1	A. Armstrong's Axioms and Inference Rules	3
2.2	B. Minimal (Canonical) Cover	4
2.3	C. Boyce-Codd Normal Form (BCNF)	5
2.4	D. Lossless Decomposition – In Depth	7
2.5	E. Dependency Preservation – In Depth	7
2.6	F. Fourth Normal Form (4NF) and Multivalued Dependencies – Awareness Level	8
2.7	G. Fifth Normal Form (5NF) – Awareness Level Only	9
2.8	H. Advanced ER Model: Participation, Weak Entities, Generalization/Specialization	10
2.9	I. Non-Binary Lossless Decomposition – A Worked Chase Example	12
2.10	J. Non-Binary Dependency Preservation – A Worked Projection Example	13
2.11	K. Formal Proofs	13
2.12	L. ER Design: From an English Description to a Relational Schema	14
3	What Comes Later — Do Not Study Now	16
4	Guided Practice (10 Questions – hints, then full solutions)	17
4.1	Armstrong's Axioms (2 questions)	17
4.2	Minimal Cover (2 questions)	17
4.3	BCNF and Decomposition (3 questions)	17
4.4	Lossless & Dependency Preservation (2 questions)	18
4.5	ER Mapping (1 question)	18
5	Guided Practice – Full Worked Solutions	19
6	Independent Practice (16 Questions)	21
6.1	Armstrong's Axioms (2 questions)	21
6.2	Minimal Cover (3 questions)	21
6.3	BCNF (4 questions)	21
6.4	Lossless / Dependency Preservation (3 questions)	22
6.5	4NF / MVD (2 questions)	22
6.6	ER Model (2 questions)	22
7	Timed Mini-Test (10 Questions – 18 minutes)	23
8	Daily Quiz (5 Questions)	24
9	Complete Answer Key	25
9.1	Independent Practice – Answers	25
9.2	Timed Mini-Test – Answers	25
9.3	Daily Quiz – Answers	26
10	Scoring and Diagnostic Interpretation	27
11	Revision Checklist	28
12	One-Page Cheat Sheet	30

1 Learning Objectives

By the end of this session you should be able to:

- State and apply Armstrong’s three axioms (reflexivity, augmentation, transitivity) and the three derived rules (union, decomposition, pseudotransitivity).
- Compute a **minimal (canonical) cover** of a small FD set: split RHS, remove extraneous LHS attributes, remove redundant FDs.
- Test whether a relation is in **BCNF**, and explain precisely how the BCNF condition differs from the 3NF condition.
- Apply the standard **BCNF decomposition algorithm** to fix a BCNF violation.
- Verify **losslessness** and **dependency preservation** for a decomposition with more rigor than Day 2’s recognition-level check, including projecting FDs onto a decomposed schema.
- Explain, at an awareness level, what a **multivalued dependency** is, why it drives **4NF**, and why **5NF** exists (without needing to execute full chase/tableau proofs).
- Apply weak-entity, participation-constraint, and generalization/specialization concepts in ER modeling, and map a small ER diagram to relational schemas correctly.
- Correctly distinguish a verified previous-year question (GATE or IOCL) from originally-written practice material, using the labels defined on the title page.

Why this matters for IOCL CBT

BCNF and the lossless/dependency-preservation distinction are the natural “next step” after Day 2’s 1NF–3NF work, and they are where PSU papers most often test whether a candidate truly understands normalization rather than having memorized definitions. ER-to-relational mapping is a classic, fast, high-confidence question type once the mapping rules are drilled. 4NF and 5NF appear rarely and briefly in PSU CBTs (if at all) – they get an awareness-level treatment here so you can recognize and eliminate wrong options, not derive full proofs under time pressure.

2 Concepts and Mental Models

Common Trap

Honest note on IOCL PYQs (read this before the rest of Section 2): A dedicated search was carried out for genuine IOCL Grade-A/Engineer CS/IT previous-year questions on Armstrong's axioms, minimal cover, BCNF, lossless/dependency preservation, 4NF/5NF, and ER design. IOCL does not publicly release official past CBT papers, and every "IOCL question bank" found during this search was a coaching-site reconstruction whose exact wording, year, and correctness could not be independently confirmed against an official source. **No question in this workbook is labeled [VERIFIED IOCL PYQ –], because none could be reliably verified.** Where a coaching site specifically claimed a question was from an IOCL paper, that claim itself could not be checked either, so even the [IOCL-REPORTED / SOURCE-UNVERIFIED] label is not used here to avoid implying a specific unverifiable claim exists. Every PYQ below is therefore a [VERIFIED GATE PYQ –] question (re-solved independently in this workbook), or clearly marked [PSU/IOCL-PATTERN PRACTICE] / [GATE-STYLE PRACTICE] where no real PYQ was reliably available for that subtopic. This is exactly what the sourcing policy for this workbook requires: report what was actually found, and never fabricate a previous-year question.

2.1 A. Armstrong's Axioms and Inference Rules

What: Armstrong's axioms are a sound and complete set of rules for deriving every FD implied by a given FD set F – i.e., for computing F^+ , the closure of F itself (not to be confused with the attribute closure X^+ from Day 2, though the two are related).

The three primary axioms:

- **Reflexivity:** If $Y \subseteq X$, then $X \rightarrow Y$. (Generates trivial FDs.)
- **Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z .
- **Transitivity:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Three derived rules (provable from the three above, but useful to know directly):

- **Union:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Decomposition:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudotransitivity:** If $X \rightarrow Y$ and $YZ \rightarrow W$, then $XZ \rightarrow W$.

Mental Model

Reflexivity gives you dependencies "for free" from set containment. Augmentation lets you "pad" both sides with the same extra attributes. Transitivity chains dependencies together. Union and Decomposition are why you can freely split or merge a same-LHS FD's RHS attributes (this is exactly what Day 2's minimal-cover-style "split the RHS" step relies on).

Worked example – deriving a new FD with the axioms

Given $A \rightarrow C$ and $B \rightarrow D$ on $R(A, B, C, D)$, prove $AB \rightarrow CD$ using only Armstrong's axioms.

1. $A \rightarrow C$ (given)
2. $AB \rightarrow CB$ (Augmentation of (1) with B)
3. $B \rightarrow D$ (given)

4. $BC \rightarrow DC$, i.e. $CB \rightarrow CD$ (Augmentation of (3) with C)
 5. $AB \rightarrow CD$ (Transitivity of (2) and (4), since $AB \rightarrow CB$ and $CB \rightarrow CD$)
 This is exactly the reasoning that justifies the “Union” shortcut you already use informally in closure calculations.

PYQ Checkpoint

[**VERIFIED GATE PYQ – GATE CSE 2024, Set 2**] [FD

Fundamentals][Counting][**[H]**]

A functional dependency $X \rightarrow Y$ is called *useful* if $X \neq \emptyset$, $Y \neq \emptyset$, and $X \cap Y = \emptyset$. For a relation R with 4 attributes, how many distinct useful functional dependencies are possible?

Solution. Every attribute independently falls into exactly one of three buckets: in X , in Y , or in neither – giving $3^4 = 81$ total assignments. Subtract the assignments where $X = \emptyset$ (each attribute is in Y or neither: $2^4 = 16$ ways) and where $Y = \emptyset$ ($2^4 = 16$ ways), then add back the case counted twice ($X = Y = \emptyset$: 1 way), by inclusion–exclusion: $81 - 16 - 16 + 1 = \mathbf{50}$.

Common Trap

Reflexivity only ever produces *trivial* FDs ($Y \subseteq X$) – it can never be used to justify a genuinely new, non-trivial dependency. Also: Augmentation pads *both* sides with the *same* Z – a common slip is augmenting only one side.

2.2 B. Minimal (Canonical) Cover

What: A minimal cover F_c of an FD set F is an equivalent set of FDs (same closure F^+) that is “as simple as possible”: every RHS is a single attribute, no FD is redundant, and no attribute on any LHS is extraneous (removable without changing what the set implies).

Why it matters: Minimal cover is the standard starting point for 3NF synthesis algorithms and for cleanly reasoning about candidate keys – PSU/GATE questions use it to test whether you can simplify an FD set without changing its meaning.

Mental Model

“Say everything the original FDs said – using the fewest, simplest FDs possible, with nothing left that could be deleted without losing information.”

Algorithm

1. **Singleton RHS:** Replace every $X \rightarrow Y_1 Y_2 \dots Y_n$ with separate FDs $X \rightarrow Y_1$, $X \rightarrow Y_2$, $\dots$ (Decomposition rule). Remove exact duplicates.
2. **Remove extraneous LHS attributes:** For each FD $X \rightarrow A$ and each attribute $B \in X$, check whether $A \in (X - B)^+$ (closure computed using the *current* FD set). If yes, B is extraneous – drop it from X .
3. **Remove redundant FDs:** For each remaining FD $X \rightarrow A$, check whether $A \in X^+$ using the *other* FDs only (i.e., the set minus this FD). If yes, this FD is implied by the rest – drop it.

Worked example

$R(A, B, C)$, $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$. Find a minimal cover.

Step 1 (split RHS, dedupe): $A \rightarrow BC$ splits into $A \rightarrow B, A \rightarrow C$. This $A \rightarrow B$ duplicates the existing one. Result: $F_1 = \{A \rightarrow B, A \rightarrow C, B \rightarrow C, AB \rightarrow C\}$.

Step 2 (extraneous LHS): Only $AB \rightarrow C$ has a composite LHS. Is A extraneous? Check $C \in (AB - A)^+ = B^+$ using F_1 : $B \rightarrow C$ fires directly, so $B^+ = \{B, C\}$, and C IS in there $\Rightarrow A$ is extraneous. Drop A : $AB \rightarrow C$ becomes $B \rightarrow C$ (a duplicate of the existing $B \rightarrow C$). $F_2 = \{A \rightarrow B, A \rightarrow C, B \rightarrow C\}$.

Step 3 (redundant FDs): Is $A \rightarrow B$ redundant? Using $\{A \rightarrow C, B \rightarrow C\}$ only: $A^+ = \{A, C\}$ – B is NOT in there, so **not** redundant, keep it. Is $A \rightarrow C$ redundant? Using $\{A \rightarrow B, B \rightarrow C\}$ only: $A^+ = \{A, B\}$, then $B \rightarrow C$ fires: $A^+ = \{A, B, C\}$ – C IS in there, so $A \rightarrow C$ **is** redundant (derivable transitively). Drop it. Is $B \rightarrow C$ redundant? Using $\{A \rightarrow B\}$ only: $B^+ = \{B\}$ – C is not there, keep it.

Minimal cover: $\{A \rightarrow B, B \rightarrow C\}$.

Common Trap

The extraneous-attribute test and the redundant-FD test both use *closure*, but over *different* FD sets – extraneous-attribute uses the *current* (not-yet-fully-reduced) set, while redundant-FD uses the *other* FDs with the one under test removed entirely. Mixing these two checks up is the single most common minimal-cover mistake. Also: a minimal cover is usually *not unique* – a differently-ordered reduction can yield a different (but equally valid) minimal cover.

PYQ Checkpoint

[GATE-STYLE PRACTICE] [Minimal Cover][[M]]

No reliably verifiable GATE or IOCL previous-year question on minimal cover could be confirmed against a documented source during research for this workbook, so an original GATE-style practice question is used here instead.

$R(A, B, C, D)$, $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$. Is this FD set already a minimal cover? Justify briefly.

Solution. RHS is already single-attribute throughout (Step 1 needs no change). Step 2: only $AB \rightarrow C$ has a composite LHS. Is A extraneous? $(AB - A)^+ = B^+$: no FD has LHS $\subseteq \{B\}$, so $B^+ = \{B\}$ – $C \notin B^+$, so A is *not* extraneous. Is B extraneous? $(AB - B)^+ = A^+$: no FD has LHS $\subseteq \{A\}$ alone, so $A^+ = \{A\}$ – $C \notin A^+$, so B is *not* extraneous either. Step 3: check each FD for redundancy using the other two – none of $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ can be derived from the remaining two (each introduces an attribute the others cannot reach). **Yes, F is already a minimal cover.**

2.3 C. Boyce-Codd Normal Form (BCNF)

Definition: R is in BCNF if for *every* non-trivial FD $X \rightarrow A$ in F^+ , X is a **super key** of R . Unlike 3NF, there is **no exception for A being prime**.

Mental Model

3NF says “ X is a super key, *or* A is prime.” BCNF deletes the “or A is prime” escape clause entirely. BCNF is therefore always at least as strict as 3NF: every relation in BCNF is automatically in 3NF, but not the reverse.

Worked example – in 3NF but NOT in BCNF

$R(A, B, C)$, FDs: $AB \rightarrow C, C \rightarrow A$.

Candidate keys: B never appears on any RHS $\Rightarrow B$ is in every key. $(AB)^+$: $AB \rightarrow C$ gives $\{A, B, C\} = R \Rightarrow$ super key; minimal ($A^+ = \{A\}$, $B^+ = \{B\}$) $\Rightarrow$ **candidate key**. $(BC)^+$: $C \rightarrow A$ fires ($C \subseteq \{B, C\}$): add $A \rightarrow \{A, B, C\} = R \Rightarrow$ super key; minimal ($B^+ = \{B\}$, $C^+ = \{A, C\}$, neither is all of R) $\Rightarrow$ **candidate key**. Candidate keys: AB and BC . Prime attributes: A, B, C (all prime – their union is everything).

3NF check: $C \rightarrow A$ – is C a super key? $C^+ = \{A, C\} \neq R$, so no. Is A prime? **Yes** (A is part of the key AB) $\Rightarrow$ the 3NF condition (“ X super key OR A prime”) is satisfied by the “ A prime” branch. $AB \rightarrow C$ trivially satisfies 3NF (AB is itself a super key). **R is in 3NF.**

BCNF check: $C \rightarrow A$ – is C a super key? **No** ($C^+ \neq R$). BCNF has no prime-attribute exception, so this alone is enough: **R is NOT in BCNF.**

BCNF Decomposition Algorithm

If R violates BCNF via some non-trivial FD $X \rightarrow A$ where X is not a super key:

1. Compute X^+ (using all of F).
2. Decompose R into $R_1 = X^+$ and $R_2 = (R - X^+) \cup X$.
3. Recheck R_1 and R_2 against BCNF; repeat on any piece that still violates it.

Applying this to the worked example above with the violating FD $C \rightarrow A$: $C^+ = \{A, C\}$, so $R_1 = \{A, C\}$ and $R_2 = (\{A, B, C\} - \{A, C\}) \cup \{C\} = \{B, C\}$. Both $R_1(A, C)$ and $R_2(B, C)$ are now trivially in BCNF (each has a single-attribute-determining-the-rest structure, or no non-trivial FD at all).

PYQ Checkpoint 1

[VERIFIED GATE PYQ – GATE CSE 2025, Set 1] [BCNF, Lossless Decomposition][Multi-Select][[M]]

Consider a relational schema $team(name, city, owner)$, with FDs

$\{name \rightarrow city, name \rightarrow owner\}$. $team$ is decomposed into $t_1(name, city)$ and $t_2(name, owner)$. Which of the following statement(s) is/are TRUE?

(A) $team$ is NOT in BCNF. (B) t_1 and t_2 are in BCNF. (C) The decomposition is a lossless join. (D) $team$ is NOT in 3NF.

Solution. $name \rightarrow city$ and $name \rightarrow owner$ together give $name^+ = \{name, city, owner\} = R$, so $name$ alone is already a super key – in fact the *only* candidate key, since it’s the only attribute that appears as a determinant. Every non-trivial FD in $team$ has $name$ (a super key) as its LHS, so $team$ **IS already in BCNF** (hence also in 3NF) – (A) and (D) are FALSE. In $t_1(name, city)$, $name$ determines the only other attribute, so $name$ is a key of t_1 – BCNF holds; same reasoning for t_2 – **(B) is TRUE**. The common attribute of the decomposition is $name$, which is a candidate key of *both* t_1 and t_2 , so joining them back reproduces $team$ exactly – **(C) is TRUE. Correct answer: (B) and (C).**

Common Trap

Trap 1: A relation can already be in BCNF even though it “looks like” it should be decomposed – always compute candidate keys first, as PYQ Checkpoint 1 shows. **Trap 2:** Unlike 3NF, a lossless *and* dependency-preserving decomposition into BCNF is not always possible – achieving BCNF can force you to give up dependency preservation (see Section 2.4). **Trap 3:** “A relation with only two attributes is always in BCNF” is TRUE – with only two attributes $\{X, Y\}$, any non-trivial FD must be $X \rightarrow Y$ or $Y \rightarrow X$, and either way the determinant alone already reaches the whole relation, so it’s automatically a super key.

2.4 D. Lossless Decomposition – In Depth

Day 2 gave the 2-relation recognition test: a decomposition of R into R_1, R_2 is lossless iff the common attribute set $R_1 \cap R_2$ is a super key of R_1 or of R_2 (equivalently, $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$ holds in F^+). That test is exact and sufficient for any 2-way decomposition – it is not an approximation.

What if R is split into 3 or more pieces? The general test uses a construction called the **chase** (built on a *tableau*): you build a table with a row per decomposed relation, mark which columns are “known” to match across rows, and repeatedly apply the FDs to force cells to become equal; the decomposition is lossless iff this process produces a row that is entirely unmarked (i.e., matches the original relation exactly).

Mental Model

Full multi-relation chase constructions are GATE-depth material and are **not required for IOCL CBT clearance** – treat this paragraph as awareness only, so you recognize the term if a question mentions it, not as something to execute under exam time pressure. In practice, most PSU/CBT-level lossless-decomposition questions are 2-way, where Day 2’s simpler common-attribute test is both correct and fast.

Common Trap

A very common trap: assuming that because *each individual piece* of a multi-way decomposition is in BCNF, the *whole decomposition* must be lossless. Losslessness is a property of how the pieces relate to each other (via the common-attribute test, or the chase for 3+ pieces) – it is not implied by each piece independently being well-normalized.

2.5 E. Dependency Preservation – In Depth

FD projection: Given a decomposition of R into $R_1, \dots, R_n$, the **projection** of F onto R_i , written $\pi_{R_i}(F)$, is the set of all FDs $X \rightarrow Y$ in F^+ such that $X \cup Y \subseteq R_i$. A decomposition preserves dependencies if the union of all these projected FDs implies every FD in the original F (equivalently, implies F^+) – **without ever needing to join the decomposed relations back together**.

Mental Model

Practical shortcut for CBT-level questions: check whether *every original FD’s full set of attributes* (both LHS and RHS) lands entirely inside *one* decomposed relation. If so, that FD is trivially preserved. Only when an original FD’s attributes are split across multiple pieces do you need to check whether it’s still *derivable* from the projected FDs.

PYQ Checkpoint

[VERIFIED GATE PYQ – GATE CSE 2019] [BCNF, Lossless & Dependency Preservation Together][[H]]

Let $F = \{QR \rightarrow S, R \rightarrow P, S \rightarrow Q\}$ hold on $X(P, Q, R, S)$. (X is not in BCNF.) X is decomposed into $Y(P, R)$ and $Z(Q, R, S)$. Consider:

I. Both Y and Z are in BCNF. II. The decomposition of X into Y, Z is dependency preserving and lossless.

Which statement(s) is/are correct?

Solution. *Candidate keys of X :* R never appears on any RHS, so R is in every key. $(QR)^+$: $QR \rightarrow S$ adds S , then $R \rightarrow P$ adds P : $\{P, Q, R, S\} = X \Rightarrow QR$ is a candidate key. $(SR)^+$: $S \rightarrow Q$ adds Q , then $R \rightarrow P$ adds P : also all of $X \Rightarrow SR$ is also a candidate key. Prime: Q, R, S . Non-prime: P .

Statement I: In $Y(P, R)$: only FD is $R \rightarrow P$, and R (single attribute) is a key of $Y \Rightarrow Y$ is in BCNF. In $Z(Q, R, S)$: FDs present are $QR \rightarrow S$ and $S \rightarrow Q$. Candidate keys of Z are QR and SR (same reasoning as above, restricted to Z 's attributes). Check $S \rightarrow Q$: is S alone a super key of Z ? S^+ (within Z) = $\{S, Q\} \neq \{Q, R, S\}$ – **no**, so Z is **NOT** in BCNF.

Statement I is FALSE.

Statement II: *Lossless:* common attribute is R ; $R \rightarrow P$ means R alone is a key of Y , so the common attribute is a super key of $Y \Rightarrow$ lossless. *Dependency preserving:* $R \rightarrow P$'s attributes lie entirely in Y ; $QR \rightarrow S$ and $S \rightarrow Q$'s attributes lie entirely in Z – every original FD is captured within one piece, no rejoin needed $\Rightarrow$ dependency preserving. **Statement II is TRUE.**

Correct answer: II only. (This is also a direct illustration of the Section 2.3 trap: X was not in BCNF, and forcing a lossless, dependency-preserving decomposition here still left one piece, Z , short of BCNF – exactly the “not always possible together” trade-off.)

2.6 F. Fourth Normal Form (4NF) and Multivalued Dependencies – Awareness Level

Multivalued dependency (MVD): Written $X \twoheadrightarrow Y$, it means: for a fixed value of X , the set of Y -values associated with it is independent of the values of every other attribute in R . An MVD always involves at least 3 attributes conceptually (the determinant, the “multivalued” side, and “the rest”).

Classic example: A course can have several independent instructors and several independent (unrelated) textbooks. Storing (Course, Instructor, Textbook) as one flat relation forces every instructor to be paired with every textbook for that course – pure redundancy, even though Instructor and Textbook have nothing to do with each other.

4NF: R is in 4NF if it is in BCNF, and every non-trivial MVD $X \twoheadrightarrow Y$ has X as a super key.

Mental Model

Every FD is also an MVD (a “single-valued” MVD), but not every MVD is an FD. 4NF is what you get when you demand the BCNF condition (X super key) for MVDs too, not just FDs. Fix: split the independent multivalued facts into separate tables, each keyed on the determinant.

Worked example

$R(\text{Course}, \text{Instructor}, \text{Textbook})$ with MVDs $\text{Course} \twoheadrightarrow \text{Instructor}$ and $\text{Course} \twoheadrightarrow \text{Textbook}$ (independent of each other). Not in 4NF (Course is not a super key of R , since R 's only candidate key is the full $\{\text{Course}, \text{Instructor}, \text{Textbook}\}$). Fix: decompose into $R_1(\text{Course}, \text{Instructor})$ and $R_2(\text{Course}, \text{Textbook})$ – both trivially in 4NF.

PYQ Checkpoint

[PSU/IOCL-PATTERN PRACTICE] [4NF, MVD Recognition][[M]]

No reliably verifiable GATE or IOCL previous-year question on multivalued dependencies / 4NF could be confirmed against a documented source during research for this workbook, so

an original PSU-pattern practice question is used here instead.

$R(\text{Employee}, \text{Skill}, \text{Language})$ stores every skill an employee has and every language they speak, with skills and languages independent of each other. Is R in 4NF? If not, decompose it.

Solution. $\text{Employee} \twoheadrightarrow \text{Skill}$ and $\text{Employee} \twoheadrightarrow \text{Language}$ are independent non-trivial MVDs, and Employee alone is not a super key of R (the only candidate key is the full attribute set). R is **NOT** in 4NF. Decompose into $R_1(\text{Employee}, \text{Skill})$ and $R_2(\text{Employee}, \text{Language})$.

Second worked example – BCNF first, THEN 4NF

$R(\text{Employee}, \text{Dept}, \text{Project})$: FD $\text{Employee} \rightarrow \text{Dept}$, and independent MVD $\text{Employee} \twoheadrightarrow \text{Project}$ (an employee works on several unrelated projects, regardless of department). Since Employee does not determine Project , the only candidate key is $\{\text{Employee}, \text{Project}\}$.

Check BCNF first: $\text{Employee} \rightarrow \text{Dept}$ – is Employee a super key?

$\text{Employee}^+ = \{\text{Employee}, \text{Dept}\} \neq R$ (missing Project) $\Rightarrow$ **NOT a super key** $\Rightarrow R$ **isn't even in BCNF yet**. Decompose on this FD first: $R_1(\text{Employee}, \text{Dept})$ and $R_2(\text{Employee}, \text{Project})$.

Now check 4NF on the pieces: $R_1(\text{Employee}, \text{Dept})$ has only 2 attributes – automatically BCNF, and no room for a non-trivial 3-attribute-style MVD, so 4NF holds trivially.

$R_2(\text{Employee}, \text{Project})$ likewise has only 2 attributes – the MVD $\text{Employee} \twoheadrightarrow \text{Project}$ is trivial within R_2 (it's the whole relation), so 4NF holds trivially there too.

Common Trap

Always clear BCNF violations before worrying about 4NF/MVDs. A plain FD violating BCNF (as in the example above) will keep re-appearing as a symptom that looks MVD-related if you try to fix the MVD first – fix ordinary FD/BCNF issues first, then re-examine what's left for multivalued dependencies.

2.7 G. Fifth Normal Form (5NF) – Awareness Level Only

What: 5NF (Project-Join Normal Form) addresses **join dependencies** that are not implied by any candidate key – cases where a relation must be split into *three or more* pieces (not just two) to eliminate a specific kind of redundancy, and where any 2-way split would lose information. This typically arises from complex multi-way business constraints (e.g., a three-way agent/company/product relationship where the valid combinations can't be derived from pairwise data alone).

Common Trap

5NF questions are rare in PSU CBTs and essentially never require you to construct a join dependency from scratch. The only realistic CBT-level ask is definitional: 5NF is the final step in the classical normalization hierarchy, and a relation in 5NF has no “lossless 3-or-more-way join” anomaly remaining. Do not spend study time deriving 5NF decompositions by hand – recognize the term and move on.

2.8 H. Advanced ER Model: Participation, Weak Entities, Generalization/Specialization

Participation constraint: whether *every* instance of an entity set must participate in a relationship (**total participation**, drawn as a double line) or only *some* instances need to (**partial participation**, single line).

Weak entity set: an entity set with no candidate key of its own attributes alone. It is identified only by combining a **partial key** (discriminator) with the primary key of its **owner (identifying) entity**, via a total-participation **identifying relationship**.

Worked example – weak entity

Dependent(*DependentName*, *DOB*) is a weak entity owned by *Employee*(*EmpID*, ...) via an identifying relationship *HasDependent*. *Dependent*'s partial key is *DependentName* (unique only *within* one employee's dependents, not across the whole table). The full identifying key of the weak entity, once mapped to a relation, is (*EmpID*, *DependentName*).

Generalization vs. Specialization:

- **Specialization** (top-down): start from a higher-level entity (e.g., *Employee*) and split it into lower-level sub-entities with extra, specific attributes (e.g., *Engineer*, *Manager*).
- **Generalization** (bottom-up): start from several lower-level entities that share common attributes and combine them into one higher-level entity.

Both use **attribute inheritance**: a sub-entity inherits all attributes of its super-entity (never the reverse).

Mental Model

Same diagram, opposite direction of design thinking. Specialization asks “how does this general entity break down?”; generalization asks “what do these specific entities have in common?”

ER-to-Relational Mapping Rules (summary)

ER construct	Mapping rule
Strong entity	One table; entity's key becomes the table's primary key.
Weak entity	One table; primary key = owner's primary key + the weak entity's partial key. A foreign key to the owner is included.
1:1 relationship	Add the PK of one side as a FK in the other (commonly the side with total participation, to avoid NULLs), or merge into one table if both sides always exist together.
1:N relationship	Add the PK of the "1" side as a FK on the "N" side. No separate table needed.
M:N relationship	Create a separate junction table holding FKs to both participating entities (plus any relationship-specific attributes).
Multivalued attribute	Create a separate table with a FK back to the owning entity's key.
Composite attribute	Flatten into its simple component attributes in the owning table.
Generalization/ Specialization	Three standard options: (a) one table per sub-entity carrying all inherited attributes; (b) one table for the super-entity plus one table per sub-entity holding only the extra attributes (linked by the shared key); (c) a single table for everything with a "type" discriminator column and nullable sub-entity-specific columns.

PYQ Checkpoint

[GATE-STYLE PRACTICE] [ER-to-Relational Mapping][[M]]

Research for this workbook found several GATE ER-diagram questions referenced by coaching sites (e.g., involving a weak entity set and an owner entity set), but none could be reconstructed with full confidence in both exact wording and answer from publicly available material, so an original GATE-style practice question is used here instead.

An ER diagram has a strong entity *Department*(*DeptID*, *DeptName*) and a weak entity *Project*(*ProjCode*, *Budget*) identified by *Department* via a total-participation relationship *Runs* (one department runs many projects; a project belongs to exactly one department; project codes repeat across departments but are unique within a department). Map this to relational schemas.

Solution. *Department*(*DeptID*, *DeptName*). *Project* is a weak entity, so its table carries the owner's key as part of its own primary key, plus a foreign key back to the owner: *Project*(*DeptID*, *ProjCode*, *Budget*), with *DeptID* as a foreign key referencing *Department*(*DeptID*). No separate table is needed for the identifying relationship *Runs* itself – it is captured entirely by *DeptID* sitting inside *Project*'s own primary key.

Common Trap

A weak entity's table *always* needs the owner's primary key as part of (or all of) its own primary key – a foreign key alone, without folding it into the primary key, is not enough to correctly represent identifying dependency. Also: total participation of the weak entity in its identifying relationship is mandatory by definition (a dependent instance cannot exist without its owner) – a weak entity can never have partial participation in its *identifying* relationship, though it may have partial participation in other, non-identifying relationships.

Mental Model

Sections 2.9–2.12 are a deeper, optional extension. Everything above this point is enough for solid IOCL CBT clearance. The next four subsections go further – non-binary (3+-way) lossless and dependency-preservation checks worked in full, short formal proofs, and two complete “English description $\rightarrow$ ER design $\rightarrow$ relational schema” problems. These match GATE-level and interview-level depth. If you’re short on time, it’s safe to skip to Section 3 and come back to these later.

2.9 I. Non-Binary Lossless Decomposition – A Worked Chase Example

For a decomposition into 3 or more pieces, build a **tableau**: one row per decomposed relation, one column per attribute of R . In row i , column j : if attribute j belongs to relation R_i , write the **distinguished symbol** for that column (same symbol in every row that includes the attribute); otherwise write a **non-distinguished symbol** unique to that cell. Then repeatedly scan the FDs: whenever two rows agree on an FD’s LHS, force their RHS values equal (if either value is distinguished, the result is the distinguished symbol). The decomposition is lossless iff this process ever produces a row that is *entirely* distinguished symbols – such a row exactly reproduces a general tuple of R .

Worked example

$R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$, decomposed into $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$. Using distinguished symbols a, b, c, d for columns A, B, C, D and $\times$ (with a row/column subscript, all distinct) for non-distinguished cells:

	A	B	C	D
$R_1(A, B)$	a	b	$\times_{13}$	$\times_{14}$
$R_2(B, C)$	$\times_{21}$	b	c	$\times_{24}$
$R_3(C, D)$	$\times_{31}$	$\times_{32}$	c	d

Apply $B \rightarrow C$: Rows R_1, R_2 agree on column B (both = b). Force their C -values equal: R_1 ’s C was $\times_{13}$, R_2 ’s C is the distinguished c – so R_1 ’s C becomes c .

Apply $C \rightarrow D$: Now all three rows agree on column C ($= c$). Force their D -values equal: R_3 ’s D is the distinguished d , so R_1 ’s and R_2 ’s D both become d .

Updated R_1 row: (a, b, c, d) – **entirely distinguished symbols! The decomposition is lossless.** (Cross-check: A is a candidate key of R here, since $A^+ = \{A, B, C, D\}$, and A lies entirely within R_1 – a known shortcut theorem says any decomposition where one piece contains a full candidate key is automatically lossless, which matches the chase result.)

Practice 1 – lossless case

[GATE-STYLE PRACTICE] $R(A, B, C)$, $F = \{A \rightarrow B\}$ only, decomposed into $R_1(A, B)$, $R_2(A, C)$, $R_3(B, C)$. Lossless?

Solution. Distinguished/non-distinguished setup gives $R_1 = (a, b, \times)$, $R_2 = (a, \times, c)$, $R_3 = (\times, b, c)$. Apply $A \rightarrow B$: R_1, R_2 agree on A ($= a$); force B equal: R_1 ’s $B = b$ (distinguished), R_2 ’s B becomes b . Updated $R_2 = (a, b, c)$ – entirely distinguished. **Lossless.**

Practice 2 – a lossy case (no FDs at all)

[GATE-STYLE PRACTICE] $R(A, B, C)$, $F = \emptyset$ (no FDs hold), decomposed into $R_1(A, B)$, $R_2(B, C)$, $R_3(A, C)$. Lossless?

Solution. With no FDs, nothing can ever force a cell to become distinguished – the tableau never changes from its initial state, and every row keeps at least one non-distinguished symbol. **No row ever becomes fully distinguished – NOT lossless.** This is the classic illustration that a 3-way split with no FDs tying the pieces together loses information, even though each individual piece looks harmless on its own.

2.10 J. Non-Binary Dependency Preservation – A Worked Projection Example

Worked example

$R(A, B, C, D, E)$, $F = \{A \rightarrow B, B \rightarrow C, CD \rightarrow E\}$, decomposed into $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D, E)$.

Compute each projection: $\pi_{R_1}(F)$: which FDs of F^+ fit entirely inside $\{A, B\}$? $A \rightarrow B$ does directly. $F_1 = \{A \rightarrow B\}$. $\pi_{R_2}(F)$: fits inside $\{B, C\}$? $B \rightarrow C$ does directly. $F_2 = \{B \rightarrow C\}$. $\pi_{R_3}(F)$: fits inside $\{C, D, E\}$? $CD \rightarrow E$ does directly (checking further: C^+ using all of F is just $\{C\}$, so nothing extra is derivable within $\{C, D, E\}$). $F_3 = \{CD \rightarrow E\}$.

$F' = F_1 \cup F_2 \cup F_3 = \{A \rightarrow B, B \rightarrow C, CD \rightarrow E\} = F$ itself $\Rightarrow F'$ **trivially implies every FD in F – dependency preserving.**

Is it also lossless? The only candidate key of R is $\{A, D\}$ (verify: A, D never appear on any RHS, so both are forced into every key; $(AD)^+$: $A \rightarrow B$ adds B , $B \rightarrow C$ adds C , $CD \rightarrow E$ adds E – reaches all of R). This key is *split* across R_1 (has A) and R_3 (has D) – no single piece contains it, so the “one piece has a full key” shortcut from Section 2.9 does not apply here. Running the full chase (Section 2.9’s method) on this exact decomposition shows it **terminates without a fully-distinguished row – this decomposition is dependency-preserving but NOT lossless.** This is a genuine, chase-verified illustration that the two properties really are independent, even for the same non-binary decomposition – not just a Day 2 slogan.

2.11 K. Formal Proofs

Proof of the Union rule

Claim: If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.

1. $X \rightarrow Y$ (given). 2. $X \rightarrow XY$ (Augmentation of 1 with X ; $X \cup X = X$).
3. $X \rightarrow Z$ (given). 4. $XY \rightarrow YZ$ (Augmentation of 3 with Y).
5. $X \rightarrow YZ$ (Transitivity of 2 and 4). ■

Proof of the Decomposition rule

Claim: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.

1. $X \rightarrow YZ$ (given). 2. $YZ \rightarrow Y$ (Reflexivity; $Y \subseteq YZ$).
3. $X \rightarrow Y$ (Transitivity of 1, 2). By the symmetric argument with $YZ \rightarrow Z$ (Reflexivity, since $Z \subseteq YZ$) and Transitivity with (1), $X \rightarrow Z$ too. ■

Proof of Pseudotransitivity

Claim: If $X \rightarrow Y$ and $YZ \rightarrow W$, then $XZ \rightarrow W$.

1. $X \rightarrow Y$ (given). 2. $XZ \rightarrow YZ$ (Augmentation of 1 with Z).
3. $YZ \rightarrow W$ (given). 4. $XZ \rightarrow W$ (Transitivity of 2, 3). ■

Why the common-attribute-is-a-key test guarantees losslessness (proof sketch)

Claim: If $(R_1 \cap R_2) \rightarrow R_1$ holds in F^+ , then for every legal instance r of R , $\pi_{R_1}(r) \bowtie \pi_{R_2}(r) = r$.

It always holds that $r \subseteq \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$ (every original tuple trivially reappears after projecting and rejoining) – this is true for *any* decomposition. The real content is the reverse inclusion. Take any tuple t in the join: it comes from some $t_1 \in \pi_{R_1}(r)$ and $t_2 \in \pi_{R_2}(r)$ agreeing on $R_1 \cap R_2$. So there exist $u, v \in r$ with $u[R_1] = t_1$ and $v[R_2] = t_2$; since t_1, t_2 agree on $R_1 \cap R_2$, so do u and v . Because $(R_1 \cap R_2) \rightarrow R_1$ holds on R , u and v agreeing on $R_1 \cap R_2$ forces them to agree on all of R_1 too – so $v[R_1] = u[R_1] = t_1$. Combined with $v[R_2] = t_2$, the single tuple $v \in r$ already equals t on all of $R = R_1 \cup R_2$. So $t = v \in r$, giving $\pi_{R_1}(r) \bowtie \pi_{R_2}(r) \subseteq r$. Both inclusions together give equality. ■

Dependency preservation, stated formally

Let $F_1, \dots, F_n$ be the projections of F onto $R_1, \dots, R_n$. Since every $F_i \subseteq F^+$ by construction, it's always true that $(F_1 \cup \dots \cup F_n)^+ \subseteq F^+$. **The decomposition is dependency-preserving exactly when the reverse inclusion also holds**, i.e. $(F_1 \cup \dots \cup F_n)^+ = F^+$ – equivalently, $F_1 \cup \dots \cup F_n \models F$ (every FD of the original set is logically implied by the union of the projections, with no need to ever rejoin the tables).

2.12 L. ER Design: From an English Description to a Relational Schema

The reverse direction of Section 2.8's mapping table: read a scenario, decide the entities, relationships, cardinalities, participation, and keys yourself, *then* map to relations.

Design Problem 1 – Hospital

Scenario: A hospital has doctors and patients. Each doctor has a unique *DoctorID*, a name, and a specialization. Each patient has a unique *PatientID*, a name, and an admission date. A patient can be treated by several doctors over time and a doctor treats several patients; each such treatment has a treatment date and a diagnosis. Each patient may also have several emergency contacts, but a contact only makes sense attached to one specific patient and is identified by name only within that patient's own list.

Design:

Entities (strong): *Doctor*(*DoctorID*, *Name*, *Specialization*),

Patient(*PatientID*, *Name*, *AdmissionDate*).

Entity (weak): *EmergencyContact*(*ContactName*, *Phone*), owned by *Patient*.

Relationships: *Treats* – M:N between *Doctor* and *Patient*, attributes

TreatmentDate, *Diagnosis*. *Has* – identifying relationship, *Patient* (1) to

EmergencyContact (N), total participation on the *EmergencyContact* side, partial on the *Patient* side (a patient may have zero contacts on file).

Relational mapping:

Doctor(*DoctorID*, *Name*, *Specialization*)

Patient(*PatientID*, *Name*, *AdmissionDate*)

Treats(*DoctorID*, *PatientID*, *TreatmentDate*, *Diagnosis*) – junction table for the M:N relationship, FKs to both entities.

EmergencyContact(*PatientID*, *ContactName*, *Phone*) – weak entity table; *PatientID* is both part of the primary key and a foreign key to *Patient*.

Design Problem 2 – Library

Scenario: A library has books and members. Each book has a unique *ISBN* and a title, and can have multiple authors (an author can also write multiple books, independent of who borrows what). Each member has a unique *MemberID* and a name. A member can borrow multiple books, and each borrowing has a borrow date and a due date. Each book belongs to exactly one genre, and each genre has a name and a shelf location.

Design:

Entities: *Book*(*ISBN*, *Title*), *Author*(*AuthorID*, *Name*), *Member*(*MemberID*, *Name*), *Genre*(*GenreID*, *GenreName*, *ShelfLocation*) – all strong.

Relationships: *Writes* – M:N between *Book* and *Author*, no relationship attributes.

Borrows – M:N between *Member* and *Book*, attributes *BorrowDate*, *DueDate*.

BelongsTo – 1:N from *Genre* to *Book* (each genre has many books; each book belongs to exactly one genre).

Relational mapping:

Genre(*GenreID*, *GenreName*, *ShelfLocation*)

Book(*ISBN*, *Title*, *GenreID*) – *GenreID* is a FK, since *Book* is on the “N” side of the 1:N relationship with *Genre*.

Author(*AuthorID*, *Name*)

Writes(*ISBN*, *AuthorID*) – M:N junction table, no extra attributes needed.

Member(*MemberID*, *Name*)

Borrows(*MemberID*, *ISBN*, *BorrowDate*, *DueDate*) – M:N junction table with relationship attributes.

Common Trap

The most common English-to-ER mistake: mapping a many-to-many relationship as if it were two separate 1:N relationships (giving one side a foreign key instead of building a junction table). Read the cardinality sentence carefully in both directions – “a patient can be treated by several doctors *and* a doctor treats several patients” is the tell-tale phrasing for M:N, not 1:N.

3 What Comes Later — Do Not Study Now

- Relational Algebra and Relational Calculus (operators, equivalence, query trees)
- SQL query writing and advanced SQL features
- Full multi-relation chase/tableau execution (beyond the awareness-level treatment in Section 2.4)
- Formal join-dependency derivation and hand-constructed 5NF decompositions
- Transactions, ACID properties, concurrency control, and locking protocols
- File organization, indexing (B-trees, hashing), and query optimization
- Distributed databases, NoSQL, and object-relational features

Common Trap

These are all valid, testable DBMS topics – they simply belong to Day 4 and beyond. If a mock test throws a relational-algebra or SQL question at you before Day 4, use whatever general programming/logic intuition you already have, flag it, and move on; don't stop to build the topic from scratch mid-session.

4 Guided Practice (10 Questions – hints, then full solutions)

4.1 Armstrong's Axioms (2 questions)

G1 [E] [Armstrong] Given $A \rightarrow B$ and $B \rightarrow C$ on $R(A, B, C)$, which single axiom lets you derive $A \rightarrow C$ directly?

Hint

Chaining two FDs through a common middle attribute is exactly one of the three primary axioms.

G2 [M] [Armstrong][Pseudotransitivity] Given $A \rightarrow B$ and $BC \rightarrow D$ on $R(A, B, C, D)$, derive $AC \rightarrow D$ using only the primary axioms (reflexivity, augmentation, transitivity) – do not just cite pseudotransitivity, show the two-step derivation it's built from.

Hint

Augment $A \rightarrow B$ with C first, then chain with $BC \rightarrow D$ using transitivity.

4.2 Minimal Cover (2 questions)

G3 [M] [Minimal Cover] $R(A, B, C, D)$, $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow B\}$. Is F already a minimal cover? Justify.

Hint

RHS is already singleton throughout. Check extraneous LHS attributes on $AB \rightarrow C$, then check each FD for redundancy against the other two.

G4 [M] [Minimal Cover] $R(A, B, C)$, $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$. Find a minimal cover.

Hint

No composite LHS here, so go straight to the redundancy check. One of the three FDs is derivable from the other two via transitivity.

4.3 BCNF and Decomposition (3 questions)

G5 [E] [BCNF] $R(A, B)$ with FD $A \rightarrow B$ only. Is R in BCNF?

Hint

Recall the two-attribute-relation trap from Section 2.3.

G6 [M] [BCNF] $R(P, Q, R)$, FDs: $PQ \rightarrow R, R \rightarrow Q$ (this is the same relation as Day 2's candidate-key Worked Example 2 – candidate keys $\{P, Q\}$ and $\{P, R\}$, all attributes prime). Is R in BCNF?

Hint

Check whether the determinant of $R \rightarrow Q$ is a super key – BCNF gives no exception even though Q is prime.

G7 [H] [BCNF Decomposition] Using G6's relation and the violating FD $R \rightarrow Q$, apply the BCNF decomposition algorithm to fix it.

Hint

Compute R^+ (closure of the determinant R , using all of F) first, then split into R^+ and (original attributes – R^+) $\cup \{R\}$.

4.4 Lossless & Dependency Preservation (2 questions)

G8 [M] [Lossless] $R(A, B, C)$ decomposed into $R_1(A, B)$ and $R_2(A, C)$. Given FD $A \rightarrow B$ holds. Is this decomposition lossless?

Hint

You only need the common attribute to be a key of *one* side.

G9 [H] [Dependency Preservation] Take your decomposition from G7: $R_1(Q, R)$ (carrying $R \rightarrow Q$) and $R_2(P, R)$ (carrying no non-trivial FD), from original $F = \{PQ \rightarrow R, R \rightarrow Q\}$ on $R(P, Q, R)$. Is this decomposition dependency-preserving?

Hint

Check whether $PQ \rightarrow R$'s attributes (P, Q, R) fit entirely inside one piece. If not, check whether it's still derivable from the projected FDs alone.

4.5 ER Mapping (1 question)

G10 [E] [ER Mapping] A *Department* entity has a 1:N relationship with an *Employee* entity (one department has many employees; each employee belongs to exactly one department). How is this mapped to relational schemas?

Hint

1:N relationships never need a separate junction table.

5 Guided Practice – Full Worked Solutions

G1 Solution

Transitivity. $A \rightarrow B$ and $B \rightarrow C$ directly satisfy the transitivity axiom's premise, giving $A \rightarrow C$.

G2 Solution

1. $A \rightarrow B$ (given). 2. $AC \rightarrow BC$ (Augmentation of (1) with C). 3. $BC \rightarrow D$ (given). 4. $AC \rightarrow D$ (Transitivity of (2) and (3), since $AC \rightarrow BC$ and $BC \rightarrow D$). This two-step derivation is exactly what the “Pseudotransitivity” shortcut packages together.

G3 Solution

RHS already singleton (Step 1: no change). Step 2: only $AB \rightarrow C$ has composite LHS. Is A extraneous? Check $C \in (AB - A)^+ = B^+$ using full F : no FD has LHS $\subseteq \{B\}$, so $B^+ = \{B\}$; $C \notin B^+ \Rightarrow A$ NOT extraneous. Is B extraneous? Check $C \in A^+$: no FD has LHS $\subseteq \{A\}$, so $A^+ = \{A\}$; $C \notin A^+ \Rightarrow B$ NOT extraneous. $AB \rightarrow C$ stays composite. Step 3: $AB \rightarrow C$ redundant? Using $\{C \rightarrow D, D \rightarrow B\}$: $(AB)^+ = \{A, B\}$ (neither fires) – C not reached, NOT redundant. $C \rightarrow D$ redundant? Using $\{AB \rightarrow C, D \rightarrow B\}$: $C^+ = \{C\}$ – D not reached, NOT redundant. $D \rightarrow B$ redundant? Using $\{AB \rightarrow C, C \rightarrow D\}$: $D^+ = \{D\}$ – B not reached, NOT redundant. **Yes, F is already a minimal cover.**

G4 Solution

No composite LHS – skip Step 2. Step 3: $A \rightarrow B$ redundant? Using $\{B \rightarrow C, A \rightarrow C\}$: $A^+ = \{A, C\}$ (via $A \rightarrow C$) – B not reached, NOT redundant, keep. $B \rightarrow C$ redundant? Using $\{A \rightarrow B, A \rightarrow C\}$: $B^+ = \{B\}$ (no FD has LHS $\subseteq \{B\}$) – C not reached, NOT redundant, keep. $A \rightarrow C$ redundant? Using $\{A \rightarrow B, B \rightarrow C\}$: A^+ : $A \rightarrow B$ gives $\{A, B\}$, then $B \rightarrow C$ fires: $\{A, B, C\}$ – C IS reached $\Rightarrow A \rightarrow C$ **IS redundant, remove it.**
Minimal cover: $\{A \rightarrow B, B \rightarrow C\}$.

G5 Solution

With only two attributes, any non-trivial FD ($A \rightarrow B$ here) has its determinant already reaching the whole relation, so A is automatically a super key. **Yes, R is in BCNF** (this is the general “any two-attribute relation is always in BCNF” fact).

G6 Solution

Check $R \rightarrow Q$: is R a super key? $R^+ = \{R, Q\}$ (only $R \rightarrow Q$ fires; $PQ \rightarrow R$ needs P too, which isn't present) $\neq \{P, Q, R\} \Rightarrow R$ is **NOT a super key**. BCNF gives no exception for Q being prime $\Rightarrow R(P, Q, R)$ is **NOT in BCNF** (though it IS in 3NF, since Q is prime – this satisfies 3NF's exception clause).

G7 Solution

Violating FD: $R \rightarrow Q$, determinant $X = \{R\}$. $X^+ = R^+ = \{R, Q\}$ (computed above). $R_1 = X^+ = \{Q, R\}$. $R_2 = (\{P, Q, R\} - \{Q, R\}) \cup \{R\} = \{P\} \cup \{R\} = \{P, R\}$. Check $R_1(Q, R)$: FD $R \rightarrow Q$ holds, R is a single-attribute key of $R_1 \Rightarrow$ BCNF holds. Check $R_2(P, R)$: does any non-trivial FD hold between P and R alone? $P^+ = \{P\}$, $R^+ = \{R, Q\}$ (Q isn't in R_2 , so within R_2 's attributes R reaches nothing new) – **no non-trivial FD holds within $\{P, R\}$ at all**, so R_2 trivially satisfies BCNF (vacuously – there's nothing to

violate it).

Final BCNF decomposition: $R_1(Q, R)$ and $R_2(P, R)$.

G8 Solution

Common attribute: A . Is A a key of $R_1(A, B)$? $A \rightarrow B$ holds and A is a single attribute reaching the only other attribute of $R_1 \Rightarrow$ **yes, A is a key of $R_1 \Rightarrow$ the decomposition is lossless** (only one side needs to satisfy this test).

G9 Solution

Original FDs: $PQ \rightarrow R$ and $R \rightarrow Q$. $R \rightarrow Q$'s attributes (Q, R) lie entirely within $R_1(Q, R)$ – preserved trivially. $PQ \rightarrow R$'s attributes (P, Q, R) are split across both R_1 (has Q, R) and R_2 (has P, R) – not contained in one piece. Can it still be derived from the projected FDs alone? The only projected FD available is $R \rightarrow Q$ (on R_1); R_2 has no non-trivial FD. Using just $R \rightarrow Q$, there is no way to derive that P, Q together determine R – $PQ \rightarrow R$ is **NOT derivable without rejoining**.

This decomposition is lossless (from G7's own BCNF-algorithm guarantee – verify via the common attribute R : $R \rightarrow Q$ makes R a key of R_1) but NOT dependency-preserving – a direct, worked illustration of the Section 2.3 trap that BCNF decomposition cannot always achieve both properties at once.

G10 Solution

No junction table is needed for a 1:N relationship. Add the primary key of the “1” side (*Department*) as a foreign key on the “N” side: *Department*(*DeptID*, ...), *Employee*(*EmpID*, ..., *DeptID*) with *DeptID* referencing *Department*(*DeptID*).

6 Independent Practice (16 Questions)

Attempt without referring back. Distribution: 2 Armstrong's axioms, 3 minimal cover, 4 BCNF, 3 lossless/dependency-preservation, 2 4NF/MVD, 2 ER model. Answers appear only in the Answer Key (Section 10).

6.1 Armstrong's Axioms (2 questions)

I1 [E] [Armstrong][Definition] Which axiom directly generates trivial FDs?

- (A) Transitivity (B) Augmentation (C) Reflexivity (D) Union

I2 [M] [Armstrong][Application] Given $A \rightarrow BC$ and $C \rightarrow D$, which pair of rules, applied in order, most directly derives $A \rightarrow D$?

- (A) Decomposition, then Transitivity (B) Augmentation, then Union (C) Reflexivity, then Augmentation (D) Union, then Decomposition

6.2 Minimal Cover (3 questions)

I3 [M] [Minimal Cover][Problem] $R(A, B, C)$, $F = \{A \rightarrow B, B \rightarrow A, A \rightarrow C\}$. Is $A \rightarrow C$ redundant?

- (A) Yes, redundant (B) No, not redundant (C) Cannot be determined (D) Only if $B \rightarrow C$ also holds

I4 [M] [Minimal Cover][Trap] The extraneous-attribute test for an LHS attribute uses the closure computed from:

- (A) the FD set with that FD entirely removed (B) the current FD set, with only that attribute removed from the LHS in question (C) only Armstrong's reflexivity axiom (D) the original unsplit FD set, always

I5 [H] [Minimal Cover][Trap] Which of the following is NOT a required property of a minimal cover?

- (A) Every RHS is a single attribute (B) No FD can be removed without changing the closure (C) Every LHS must be a single attribute (D) No attribute can be removed from any LHS without changing the closure

6.3 BCNF (4 questions)

I6 [E] [BCNF][Definition] BCNF differs from 3NF because BCNF:

- (A) allows composite keys (B) removes the "or A is prime" exception (C) requires 4NF first (D) only applies to weak entities

I7 [M] [BCNF][Problem] $R(A, B)$, FD $B \rightarrow A$. Is R in BCNF?

- (A) No, A is not a super key (B) Yes, B is a super key of this 2-attribute relation (C) Cannot be determined (D) No, BCNF needs at least 3 attributes

I8 [M] [BCNF][Problem] $R(W, X, Y, Z)$, candidate key = $\{W, X\}$, FDs: $WX \rightarrow Y$, $WX \rightarrow Z$, $Y \rightarrow X$. Is R in BCNF?

- (A) Yes, in BCNF (B) No – $Y \rightarrow X$ violates BCNF since Y is not a super key (C) No – $WX \rightarrow Y$ violates BCNF (D) Cannot be determined without more information

I9 [H] [BCNF][Trap] A relation is in BCNF. Which of the following MUST be true?

- (A) It has only one candidate key (B) It is also in 3NF (C) It cannot have composite candidate keys (D) It is automatically dependency-preserving under any decomposition

6.4 Lossless / Dependency Preservation (3 questions)

I10 [M] [Lossless][Problem] $R(A, B, C, D)$ decomposed into $R_1(A, B, C)$ and $R_2(A, D)$. Given FD $A \rightarrow D$ holds. Is this decomposition lossless?

(A) Yes, A is a key of R_2 (B) No, need to check R_1 too (C) Cannot determine without knowing all FDs on R_1 (D) No, decompositions with 3+ attributes on one side are never lossless

I11 [E] [Lossless][Trap] True or False: “If a decomposition is lossless, it must also preserve all functional dependencies.”

(A) True (B) False

I12 [H] [Dependency Preservation][Problem] $R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$ decomposed into $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$. Is this decomposition dependency-preserving?

(A) Yes – every original FD’s attributes fall entirely within one decomposed relation (B) No – $C \rightarrow D$ is lost (C) No – R_1 and R_2 must be rejoined first (D) Cannot be determined

6.5 4NF / MVD (2 questions)

I13 [E] [4NF][Definition] Which of these is TRUE about multivalued dependencies?

(A) Every MVD is an FD (B) Every FD is an MVD (C) MVDs only occur in relations already in BCNF (D) MVDs require exactly 2 attributes

I14 [M] [4NF][Scenario] $R(Student, Club, Sport)$, with independent MVDs $Student \twoheadrightarrow Club$ and $Student \twoheadrightarrow Sport$, and the only candidate key of R is the full attribute set. Is R in 4NF?

(A) Yes (B) No – $Student$ is not a super key, yet is the determinant of a non-trivial MVD (C) No, because clubs and sports are related to each other (D) Cannot be determined

6.6 ER Model (2 questions)

I15 [E] [ER][Definition] A weak entity’s identifying relationship with its owner must have:

(A) partial participation on the weak entity’s side (B) total participation on the weak entity’s side (C) no participation constraint at all (D) M:N cardinality, always

I16 [M] [ER Mapping][Application] An M:N relationship between entities E_1 and E_2 , with no relationship-specific attributes, maps to:

(A) a foreign key added to E_1 (B) a foreign key added to E_2 (C) a separate junction table with foreign keys to both E_1 and E_2 (D) merging E_1 and E_2 into a single table

7 Timed Mini-Test (10 Questions – 18 minutes)

Instructions: Set a timer for 18 minutes (Day 3 has more ground to cover than Days 1–2, so a slightly longer timed test). Attempt all 10 without pausing. Two of these are verified GATE PYQs, mixed in with fresh practice – try to solve both types under the same time pressure. Answers appear only in the Answer Key (Section 10).

T1 [E] [Armstrong][Definition] Augmentation states: if $X \rightarrow Y$ then:

- (A) $XZ \rightarrow Y$ (B) $XZ \rightarrow YZ$ (C) $X \rightarrow YZ$ (D) $Z \rightarrow XY$

T2 [M] [VERIFIED GATE PYQ – GATE CS 1994] [FD Reasoning][Instance vs. Schema]

An instance of a relational scheme $R(A, B, C)$ has distinct values for attribute A . Can you conclude that A is a candidate key for R ?

- (A) Yes, since all values are distinct (B) No – one instance happening to satisfy uniqueness doesn't prove the FD holds for every legal instance (C) Yes, but only if B and C are also distinct (D) Cannot be determined without knowing the normal form

T3 [M] [Minimal Cover][Concept] In minimal-cover reduction, if removing attribute B from the LHS of $AB \rightarrow C$ does NOT change A 's closure to include C , then B is:

- (A) extraneous, remove it (B) NOT extraneous, keep it (C) redundant, remove the whole FD (D) part of every candidate key

T4 [M] [BCNF][Definition] For a non-trivial FD $X \rightarrow A$ to violate BCNF:

- (A) X must not be a super key (B) A must not be prime (C) X must be composite (D) A must be a super key

T5 [H] [BCNF][Problem] $R(A, B, C)$, FDs = $\{AB \rightarrow C, C \rightarrow B\}$. Which is TRUE?

- (A) R is in BCNF (B) R is in 3NF but not BCNF (C) R is not even in 3NF (D) R has only one candidate key

T6 [M] [Lossless][Concept] Which condition guarantees a 2-way decomposition R_1, R_2 of R is lossless?

- (A) $R_1 \cap R_2 = \emptyset$ (B) $R_1 \cap R_2$ is a super key of R_1 or R_2 (C) $R_1 \cup R_2$ has fewer attributes than R (D) Every FD of R appears in R_1

T7 [M] [Dependency Preservation][Concept] Dependency preservation means:

- (A) every decomposed relation must be in BCNF (B) all original FDs must be derivable from the union of projected FDs, without rejoining (C) the decomposition must also be lossless (D) every original FD must appear verbatim, unchanged, in exactly one relation

T8 [E] [4NF][Definition] 4NF requires BCNF plus:

- (A) no non-trivial MVD unless the determinant is a super key (B) no transitive FDs (C) atomic values only (D) a single candidate key

T9 [M] [ER][Definition] Generalization is best described as:

- (A) top-down splitting of an entity into sub-entities (B) bottom-up combining of entities into a higher-level entity (C) adding a foreign key to a weak entity (D) converting an M:N relationship into 1:N

T10 [H] [VERIFIED GATE PYQ – GATE CS 1994] [BCNF][True/False with reason]

True or False, with reason: "There is always a decomposition into BCNF that is lossless and dependency preserving."

- (A) True (B) False – a lossless BCNF decomposition always exists, but dependency preservation cannot always be achieved at the same time

8 Daily Quiz (5 Questions)

Final quick check – no solutions immediately after; see Answer Key (Section 10).

Q1 [E] [Armstrong][Definition] Transitivity states: if $X \rightarrow Y$ and $Y \rightarrow Z$ then:

- (A) $X \rightarrow Z$ (B) $Z \rightarrow X$ (C) $XY \rightarrow Z$ (D) $Y \rightarrow X$

Q2 [M] [Minimal Cover][Concept] True or False: “A minimal cover of a given FD set is always unique.”

- (A) True (B) False

Q3 [E] [BCNF][Definition] BCNF requires every non-trivial FD’s determinant to be a:

- (A) prime attribute (B) super key (C) composite key (D) foreign key

Q4 [M] [Lossless vs. Dependency Preservation][Trap] Which property can fail even when a decomposition achieves BCNF?

- (A) Losslessness (B) Dependency preservation (C) Atomicity (D) Both always hold together once BCNF is reached

Q5 [E] [ER][Definition] A weak entity is identified using:

- (A) its own attributes alone (B) the owner’s primary key combined with its own partial key
(C) a randomly generated ID (D) generalization

9 Complete Answer Key

9.1 Independent Practice – Answers

- I1. (C) **Reflexivity.** $Y \subseteq X \Rightarrow X \rightarrow Y$ is always trivial.
- I2. (A) **Decomposition, then Transitivity.** $A \rightarrow BC$ decomposes to $A \rightarrow C$; then transitivity with $C \rightarrow D$ gives $A \rightarrow D$.
- I3. (B) **No, not redundant.** Using $\{A \rightarrow B, B \rightarrow A\}$ only: $A^+ = \{A, B\} - C$ is never reached, so $A \rightarrow C$ cannot be dropped.
- I4. (B). The extraneous-attribute test removes just one attribute from one FD's LHS and recomputes closure using the rest of the *current* set – it does not remove the whole FD (that's the separate redundant-FD test).
- I5. (C). A minimal cover's LHS attributes need not be single – only the RHS must be a single attribute. Composite LHS (like $AB \rightarrow C$) is perfectly normal in a minimal cover.
- I6. (B). BCNF drops 3NF's "or A is prime" escape clause – every non-trivial determinant must be a super key, full stop.
- I7. (B). With only 2 attributes, $B \rightarrow A$ makes B reach the whole relation, so B is automatically a super key – any 2-attribute relation with a non-trivial FD is in BCNF.
- I8. (B). $Y \rightarrow X$: $Y^+ = \{X, Y\} \neq R - Y$ is not a super key, so this FD violates BCNF regardless of X 's prime status (BCNF has no such exception). $WX \rightarrow Y$ and $WX \rightarrow Z$ are both fine since WX is the candidate key itself.
- I9. (B). BCNF always implies 3NF (BCNF is strictly stricter). (A) and (C) are false – BCNF relations can have multiple, composite candidate keys. (D) is false – see Section 2.3's Trap 2 and the GATE CSE 2019 PYQ Checkpoint.
- I10. (A). $A \rightarrow D$ with A a single attribute reaching the only other attribute of $R_2(A, D)$ makes A a key of R_2 – sufficient on its own for losslessness.
- I11. (B) **False.** Losslessness and dependency preservation are independent properties (reinforced repeatedly in this workbook, e.g. the G7/G9 worked pair and the GATE CSE 2019 PYQ).
- I12. (A). $A \rightarrow B$'s attributes are both in R_1 ; $B \rightarrow C$'s are both in R_2 ; $C \rightarrow D$'s are both in R_3 – every original FD is captured within a single piece.
- I13. (B). Every FD $X \rightarrow Y$ is trivially also an MVD $X \twoheadrightarrow Y$ (the Y -values for a fixed X are a single value, hence trivially independent of everything else). The reverse does not hold in general.
- I14. (B). *Student* determines both *Club* and *Sport* via non-trivial MVDs but is not itself a super key (the only candidate key is the whole relation) – this violates 4NF.
- I15. (B). A dependent instance cannot exist without its owner, so participation in the identifying relationship is mandatory (total) on the weak entity's side.
- I16. (C). M:N relationships always need a separate junction/bridge table holding foreign keys to both participating entity sets.

9.2 Timed Mini-Test – Answers

- T1. (B) $XZ \rightarrow YZ$. Augmentation pads *both* sides with the same Z .
- T2. (B). An FD (and hence a key) is a schema-level guarantee that must hold for every legal

instance – one instance with distinct A -values could simply be coincidence.

- T3. (B) NOT extraneous, keep it.** If dropping B loses the ability to reach C , B was actually necessary in the determinant.
- T4. (A).** BCNF violation happens precisely when the determinant X fails to be a super key – A 's prime status is irrelevant to BCNF (unlike 3NF).
- T5. (B).** Candidate keys: AB (via $AB \rightarrow C$) and AC (via $C \rightarrow B$, since $C \subseteq AC$ fires and reaches B). Both are prime-attribute-only keys, so $C \rightarrow B$ satisfies 3NF's " B is prime" exception – 3NF holds. But $C^+ = \{B, C\} \neq R$, so C is not a super key – BCNF fails.
- T6. (B).** This is exactly the Day 2 / Section 2.4 common-attribute test.
- T7. (B).** Derivability from the projected FDs is what matters – an FD can be preserved even if it doesn't appear verbatim, as long as it's still logically implied.
- T8. (A).** 4NF = BCNF + every non-trivial MVD's determinant is a super key.
- T9. (B).** Generalization combines specific entities upward into a shared general entity; specialization is the reverse, top-down, direction.
- T10. (B) False.** A lossless BCNF decomposition is always achievable, but a *simultaneously* dependency-preserving one is not always possible – illustrated directly by the GATE CSE 2019 PYQ Checkpoint in Section 2.5.

9.3 Daily Quiz – Answers

- Q1. (A)** $X \rightarrow Z$. The direct chaining rule.
- Q2. (B) False.** A different reduction order can produce a different, equally valid minimal cover.
- Q3. (B) Super key.** No prime-attribute exception in BCNF.
- Q4. (B) Dependency preservation.** BCNF guarantees a lossless decomposition exists, but not necessarily one that is also dependency-preserving.
- Q5. (B).** Owner's primary key + the weak entity's own partial key together form the identifying key.

10 Scoring and Diagnostic Interpretation

Score band	Interpretation and next action
<60%	Revisit Section 2.3 (BCNF) and Section 2.5 (dependency preservation) tonight – these carry the most weight today. Redo G5–G9 before Day 4.
60–75%	Acceptable. Log weak sub-topics (minimal cover vs. BCNF vs. dependency preservation) in the error log below.
75–85%	Good. Log individual misses, then proceed to Day 4 as scheduled.
>85%	Strong. Proceed to Day 4 (Relational Algebra, SQL, Transactions, Indexing) with confidence.

Note: This is a personal study diagnostic only, not an official IOCL cutoff or scoring benchmark. No IOCL-specific subject weightage is implied by these bands – and, as noted in Section 2, no verified IOCL previous-year question could be sourced for today’s topics, so this session leans more heavily on verified GATE PYQs than Days 1–2 did.

11 Revision Checklist

14-Item Revision Checklist – tick before moving to Day 4

- ☐ I can state all three of Armstrong's primary axioms (reflexivity, augmentation, transitivity) and all three derived rules (union, decomposition, pseudotransitivity).
- ☐ I can derive a new FD from given FDs using the axioms, showing each step.
- ☐ I can compute a minimal cover: split RHS, remove extraneous LHS attributes, remove redundant FDs – and I know these two removal checks use closure over *different* FD sets.
- ☐ I know a minimal cover is not necessarily unique.
- ☐ I can state the BCNF condition and explain exactly how it differs from 3NF (no prime exception).
- ☐ I know any 2-attribute relation with a non-trivial FD is automatically in BCNF.
- ☐ I can apply the BCNF decomposition algorithm (X^+ , then split) to fix a violation.
- ☐ I know a lossless BCNF decomposition always exists, but a simultaneously dependency-preserving one does not.
- ☐ I can verify losslessness via the common-attribute-is-a-key test, and I know the chase/tableau generalizes this for 3+-way decompositions (awareness level only).
- ☐ I can check dependency preservation by seeing whether each original FD's attributes fall within one piece, or are still derivable from the projected FDs.
- ☐ I know every FD is an MVD but not every MVD is an FD, and I can state the 4NF condition.
- ☐ I know 5NF addresses join dependencies not implied by any candidate key, at an awareness level only.
- ☐ I can identify a weak entity, its partial key, and its owner's role in the identifying relationship.
- ☐ I can map strong entities, weak entities, 1:1/1:N/M:N relationships, and generalization/specialization to relational schemas correctly.

Things I Got Wrong Today

Tag the category before logging the specifics:

- ☐ Armstrong's axioms mistake (wrong rule cited, or misapplied augmentation/transitivity)
- ☐ Minimal cover mistake (confused the extraneous-attribute test with the redundant-FD test)
- ☐ BCNF mistake (applied the 3NF prime exception where BCNF doesn't allow one)
- ☐ BCNF decomposition mistake (wrong X^+ computation, or wrong split)
- ☐ Lossless-decomposition mistake (checked the wrong side, or assumed 3+ piece losslessness without the chase)
- ☐ Dependency-preservation mistake (assumed lossless $\Rightarrow$ dependency-preserving, or vice versa)
- ☐ 4NF / MVD mistake (confused MVD with FD, or missed an independent multivalued fact)
- ☐ ER weak-entity / participation mistake
- ☐ ER-to-relational mapping mistake (wrong FK placement, or missed a junction table)

QuestionTopic / Cate- gory	My mistake	Correct rule	Revisit date
-------------------------------	------------	--------------	-----------------

12 One-Page Cheat Sheet

Armstrong's Axioms

Reflexivity: $Y \subseteq X \Rightarrow X \rightarrow Y$ (trivial).

Augmentation: $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$.

Transitivity: $X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$.

Derived: Union ($X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$); Decomposition (reverse of Union); Pseudotransitivity ($X \rightarrow Y, YZ \rightarrow W \Rightarrow XZ \rightarrow W$).

Minimal Cover – 3 Steps

1. Split every RHS to a single attribute.
 2. Remove extraneous LHS attributes: B extraneous in $X \rightarrow A$ if $A \in (X - B)^+$ (using current F).
 3. Remove redundant FDs: $X \rightarrow A$ redundant if $A \in X^+$ using the *other* FDs only.
- Not necessarily unique.

BCNF

Every non-trivial $X \rightarrow A$: X is a super key.

No prime-attribute exception (unlike 3NF).

Any 2-attribute relation with a non-trivial FD is automatically BCNF.

BCNF $\Rightarrow$ 3NF, not the reverse.

BCNF Decomposition Algorithm

Violating FD $X \rightarrow A$, X not a super key:

$R_1 = X^+$. $R_2 = (R - X^+) \cup X$.

Recurse on any piece still violating BCNF.

Lossless Decomposition

2-way: $R_1 \cap R_2$ is a super key of R_1 or R_2 .

3+-way: use the chase/tableau test (awareness level).

A lossless BCNF decomposition always exists.

Dependency Preservation

FD projection $\pi_{R_i}(F)$: FDs of F^+ whose attributes fit entirely in R_i .

Preserved if the union of all projected FDs implies every original FD – without rejoining.

Shortcut: if an FD's attributes fit in one piece, it's preserved automatically.

Lossless $\nRightarrow$ dependency-preserving, and vice versa.

BCNF cannot always get both at once – 3NF synthesis always can.

4NF / 5NF (awareness)

MVD $X \twoheadrightarrow Y$: Y -values for fixed X are independent of the rest.

Every FD is an MVD; not every MVD is an FD.

4NF: BCNF + every non-trivial MVD's determinant is a super key.

5NF: no lossless 3+-way join anomaly remains (rare in CBTs).

ER Mapping Quick Table

Strong entity $\rightarrow$ table, key = PK.

Weak entity $\rightarrow$ table, PK = owner's PK + partial key.

1:N $\rightarrow$ FK on the "N" side.

M:N $\rightarrow$ junction table with both FKs.

Multivalued attr. $\rightarrow$ separate table.

Generalization $\rightarrow$ per-subtype tables, or one shared table + discriminator.

End of Day 3 Workbook – IOCL Grade-A 2026 CS/IT Preparation

Next: Day 4 – Relational Algebra, Relational Calculus, SQL, Transactions & Concurrency, File Organization & Indexing